PROJECT WALKTHROUGH

Apartment Society Management System

How it works, how it is secured, and how to use it

Next.js · MySQL (Prisma) · MongoDB (Mongoose) · bcrypt · JWT sessions

What this document covers

- What the system does and who uses it
- Architecture: how a request travels from the browser to the two databases
- Sign-in security in detail: bcrypt password hashing, sessions, logout, throttling
- Roles and permissions, including owners, tenants and people with several flats
- Account requests and admin approval
- The data model in MySQL and MongoDB
- Key workflows, edge cases, testing and known limitations

Repository: github.com/AhidwishJavali/Apartment-society-management
Version described: main branch, 31 October 2026
Demo passwords are deliberately not printed here: see test-logins.txt or .demo-credentials.local.

Contents

1. What the system is . . . 3
Who uses it . . . 3
Technology . . . 3
2. Architecture . . . 4
What happens on every API request . . . 4
Where things live in the code . . . 5
3. Running it . . . 6
Environment variables . . . 6
Demo accounts . . . 6
Useful commands . . . 7
4. Sign-in and security in detail . . . 8
4.1 How passwords are hashed . . . 8
4.2 What happens when someone signs in . . . 9
4.3 Sessions . . . 10
4.4 Logout, password resets and revocation . . . 10
4.5 Other protections . . . 11
4.6 Requesting an account and admin approval . . . 11
5. Roles and what each person sees . . . 13
5.1 Owners and tenants . . . 13
5.2 One person with several flats . . . 14
6. The data model . . . 16
MySQL tables . . . 16
MongoDB collections . . . 16
Migrations . . . 17
7. Key workflows . . . 18
Complaints . . . 18
Maintenance billing . . . 18
Parking . . . 18
Visitors . . . 18
Notices . . . 18
8. Edge cases handled . . . 19
9. Testing and quality checks . . . 20
10. Known limitations . . . 20
Appendix: API at a glance . . . 21

1. What the system is

The Apartment Society Management System is a web application for running a housing society. Administrators manage flats, residents, staff, maintenance bills, parking and vehicles; residents see their own bills, raise complaints and read notices; maintenance staff work through the complaints assigned to them; security staff log visitors at the gate. Everyone signs in to the same site and sees only what their role allows.

Who uses it

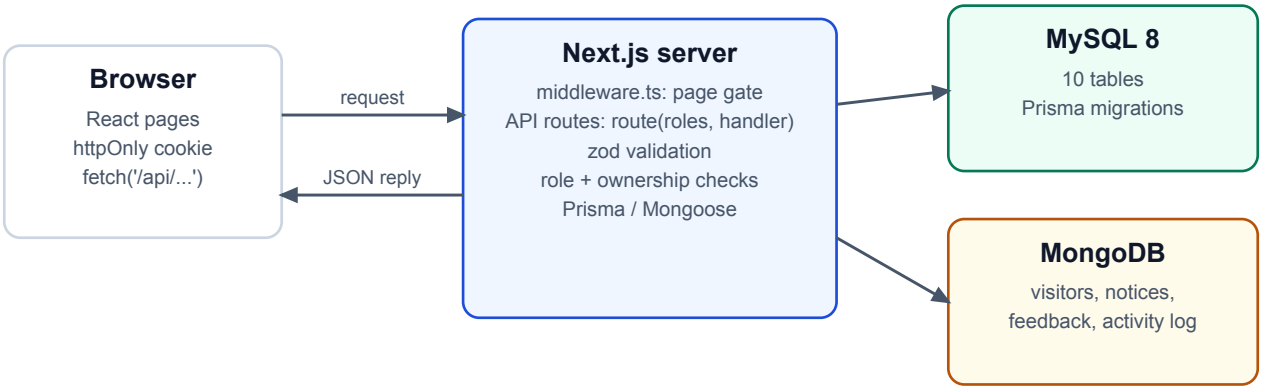
Role	Typical person	Main job in the app
Administrator	Society secretary or office	Everything: records, billing, approvals, assigning complaints, notices
Resident (owner)	Someone who owns one or more flats	Pays maintenance, sees their household, raises complaints
Resident (tenant)	Someone renting a flat	Raises complaints, parking and vehicles; no access to bills
Maintenance	Electrician, plumber, supervisor	Works on and resolves complaints assigned to them
Security	Gate guard	Registers visitor entries and exits

Technology

Layer	Technology (version in use)	Why
Web framework	Next.js 15.5 with React 19, TypeScript	Pages and API in one project; server-side checks on every request
Relational data	MySQL 8 through Prisma 6.19	Flats, people, bills and parking need foreign keys, unique rules and transactions
Document data	MongoDB through Mongoose 8.24	Visitor logs, notices, feedback and activity history are event-like and flexible
Passwords	bcrypt (bcryptjs 3.0), cost factor 12	Slow, salted, one-way hashing designed for passwords
Sessions	Signed JSON Web Tokens (jose 6.2, HS256)	Stateless cookie, re-checked against the database each request
Validation	zod	Every request body and query string is checked before use
Styling	Tailwind CSS	Responsive layout for phones and desktops

2. Architecture

The browser never talks to a database. It loads pages from the Next.js server and calls JSON endpoints under `/api/...`. Those endpoints run on the server, check who is asking and what they may do, and only then read or write MySQL or MongoDB.



Joined by MySQL integer ids

Every API request passes through these steps (src/lib/api.ts and the route handler):

1 Origin check → 2 Verify JWT → 3 Re-read account → 4 Role check → 5 Validate input → 6 Ownership → 7 Query + reply

What happens on every API request

1. **Origin check.** For anything that changes data (POST, PATCH, DELETE), the request's `Origin` header must match the site's own host. A malicious page on another site cannot submit forms on a signed-in user's behalf.
2. **Session check.** The `society_session` cookie is read and its signature and expiry verified. No valid cookie means 401.
3. **Account re-read.** The account is loaded fresh from MySQL, so a deactivated account, a revoked session or a removed flat takes effect immediately, not when the cookie expires.
4. **Role check.** Each endpoint declares which roles may call it; anyone else gets 403.
5. **Input validation.** The body and query string are parsed with zod schemas; bad input gets 400 with per-field messages.
6. **Ownership check.** The handler narrows the query to records the person may see, for example only bills of flats they own.
7. **Query and reply.** Prisma or Mongoose runs the query and the result is wrapped in a consistent `{ data, meta }` or `{ error: { code, message } }` envelope.

> **Hiding a button is never the security**
>
> The menu hides pages a role cannot use, but that is only convenience. Every endpoint repeats the role and ownership checks on the server, so typing an address directly or calling the API by hand gets the same refusal. For example, a tenant who types /payments is refused by the server and shown an explanation (section 5).

Where things live in the code

Path	Contents
src/app/(dashboard)/...	One page per module: dashboard, apartments, residents, payments, parking, complaints...
src/app/(auth)/...	Public pages: sign-in and request an account
src/app/api/...	JSON endpoints, one folder per resource
src/lib/auth.ts, session-token.ts	Password hashing, session tokens, loading the signed-in user
src/lib/api.ts	The route() wrapper that applies steps 1 to 4 and the error envelope
src/lib/rules.ts, schemas.ts	Business rules (status changes, notice audiences) and input validation
src/server/...	Shared server helpers: parking allocation, account linking, access scoping
prisma/	MySQL schema and the five versioned migrations
tests/	Unit tests and integration tests against live databases

3. Running it

You need Node.js 20.12 or newer, MySQL 8 and MongoDB 6 or newer. With Docker, `docker compose up -d` starts both databases with matching settings.

```
git clone https://github.com/AhidwishJavali/Apartment-society-management.git
cd Apartment-society-management
npm install
cp .env.example .env.local && cp .env.example .env   # then fill in the values below
npm run db:setup     # migrations + MongoDB indexes + demo data
npm run dev          # http://localhost:3000
```

Environment variables

Variable	Example	Notes
DATABASE_URL	mysql://society_user:change_me@localhost:3306/apartment_society	Must start with mysql://
MONGODB_URI	mongodb://localhost:27017/apartment_society	Must start with mongodb://
SESSION_SECRET	output of: openssl rand -base64 48	At least 32 characters; signs every session cookie

The app refuses to start an API call with a missing or malformed variable and names the problem, instead of failing later with an obscure database error.

Demo accounts

The seed creates five accounts. Each password is random, printed once when the seed runs, and saved to `.demo-credentials.local` (also copied to `test-logins.txt` in the working copy). Both files are git-ignored. The passwords are not printed in this document so it can be shared safely.

Username	Role	Linked to	Lands on
admin	Administrator	Standalone	Dashboard
resident	Resident, owner of two flats	Arjun Rao: lives in and owns A-101; also owns A-201	Dashboard
tenant	Resident, tenant	Rahul Mehta, renting A-201 from Arjun	Dashboard
maintenance	Maintenance	Suresh Kumar (Electrician)	Complaints
security	Security	Mahesh Rao (Security Guard)	Visitors

Useful commands

Command	What it does
npm run db:seed	Adds demo data; safe to re-run, never creates duplicates
npm run db:reset	Deletes all app data in both databases and re-seeds with new passwords (asks you to type reset)
npm test	21 unit tests (no database needed)
npm run test:integration	18 tests against live MySQL and MongoDB; cleans up after itself
npm run check	Lint, type-check, unit tests and production build

4. Sign-in and security in detail

4.1 How passwords are hashed

The system never stores a password. It stores a **bcrypt hash**: a fixed-length fingerprint computed from the password that cannot be turned back into it. When someone signs in, the password they type is hashed the same way and the two results are compared.

Why bcrypt, and what the settings mean

- **One-way.** There is no decrypt step. Even with full access to the database, nobody can read a password back out.
- **Salted.** Before hashing, bcrypt generates 16 random bytes (the *salt*) and mixes them in. Two people with the same password get completely different hashes, so an attacker cannot use precomputed tables or spot shared passwords.
- **Deliberately slow.** The cost factor is **12**, meaning 2^{12} = 4,096 rounds of bcrypt's expensive key setup. Measured on the development Mac, one hash takes about **0.3 seconds** (cost 10 would be about 0.07 s). That is unnoticeable when signing in once, but it makes guessing millions of passwords against a stolen database impractical.
- **Self-describing.** The algorithm, cost and salt are stored inside the hash itself, so the cost can be raised later without a schema change; old hashes keep working.

Anatomy of one stored hash (60 characters in UserAccount.password_hash):



Same password, hashed twice

This is a real example made for this document from the made-up password `example-password-123` (no account uses it). The salt differs, so the whole hash differs, yet both verify:

```
hash #1  $2b$12$/onRGqooCQCHezi19iRTLehEg6vdMK62UluoVnYXWpt6CvcizMskO
hash #2  $2b$12$DID/FnSpN.9kjfUD/WQZwecxYJeL9oOHd3/8WgqApxtjhLZmqQncS

compare('example-password-123', hash #1)  ->  true
compare('example-password-124', hash #1)  ->  false   (one character different)
```

The code

```
// src/lib/auth.ts
export const hashPassword   = (plain) => bcrypt.hash(plain, 12);
export const verifyPassword = (plain, hash) => bcrypt.compare(plain, hash);
```

Where hashing happens

Moment	What is hashed and stored
Admin creates an account	The initial password, hashed before the INSERT
Admin resets a password	The new password; the old hash is overwritten and every existing session is ended
Someone requests an account	Hashed the moment the form is received and kept in AccountRequest.password_hash; on approval that hash is copied into the new account, so the plain password never touches the database
Seed script	Each random demo password, hashed with the same cost 12

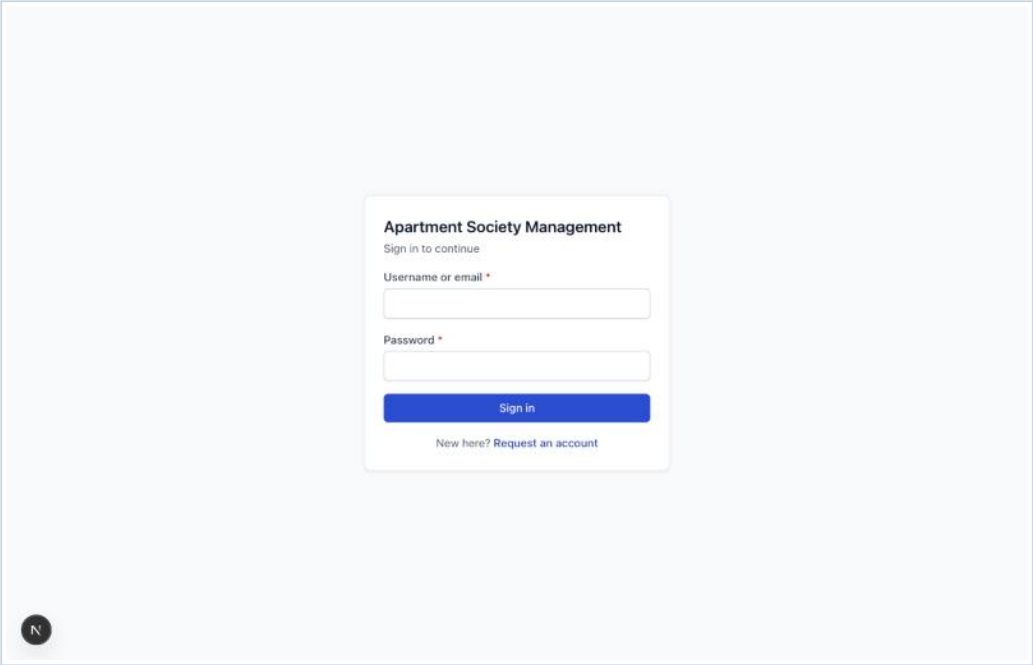
Password rules

At least 10 characters, at most 100. Plain passwords are never logged, never returned by any API, and never selected from the database: account lists explicitly leave the hash column out.

bcrypt only uses the first 72 bytes of a password, so characters beyond that do not add strength. Normal passwords are far shorter.

4.2 What happens when someone signs in

1. The form sends the username or email and password to `POST /api/auth/login`.
2. **Throttle check.** Failed attempts are counted per username/email in the MySQL table `LoginAttempt`. After **5 failures within 15 minutes** further attempts get 429 Too Many Attempts. Because the counter lives in the database, it holds across server restarts and multiple server processes.
3. The account is looked up by username or email (email is matched in lowercase).
4. **Constant-time behaviour.** A bcrypt comparison always runs, against a dummy hash if the account does not exist. Otherwise an attacker could tell real usernames from fake ones by how fast the reply comes back.
5. The account must be active; a resident must still have at least one active flat; a staff member must still be active.
6. On any failure the reply is the same generic "Invalid username/email or password", and the failure counter goes up.
7. On success the counter is cleared, `last_login_at` is recorded and a session cookie is issued.



The sign-in page. "Request an account" leads to the public sign-up form (section 4.6).

4.3 Sessions

After sign-in the browser holds one cookie, `society_session`, containing a **JSON Web Token** signed with `SESSION_SECRET` using HMAC-SHA256 (HS256).

Property	Value	Effect
Contents	user id, role, issued-at time in milliseconds, expiry	No password, hash or personal data inside
Signature	HS256 with SESSION_SECRET (32+ characters)	Any edit to the token, such as changing the role to ADMIN, breaks the signature and it is rejected
Lifetime	8 hours	Then the person signs in again
httpOnly	Yes	Page scripts cannot read the cookie, so injected JavaScript cannot steal it
SameSite	Lax	Not sent on cross-site form posts; together with the Origin check this blocks request forgery
Secure	Yes in production	Only sent over HTTPS

The token is signed, not encrypted: anyone holding it could read the user id and role, which is why it holds nothing sensitive. What nobody can do without the secret is create or alter one. The server also never trusts the role inside the token on its own; it re-reads the account from MySQL on every request (step 3 in section 2).

4.4 Logout, password resets and revocation

Plain JWTs cannot normally be cancelled before they expire. This system adds a column, `UserAccount.sessions_valid_after`. Any token issued at or before that moment is rejected.

- **Sign out** sets it to now: the cookie is cleared and even a copied cookie stops working, on every device.
- **Password reset** by an admin sets it too, so a session opened with the old password ends.
- **Deactivating** an account, or a resident's last flat, locks the login on the very next request.

- Tokens carry their issue time in **milliseconds**; an earlier version used whole seconds and a test caught that a sign-out in the same second as the sign-in did not revoke it.

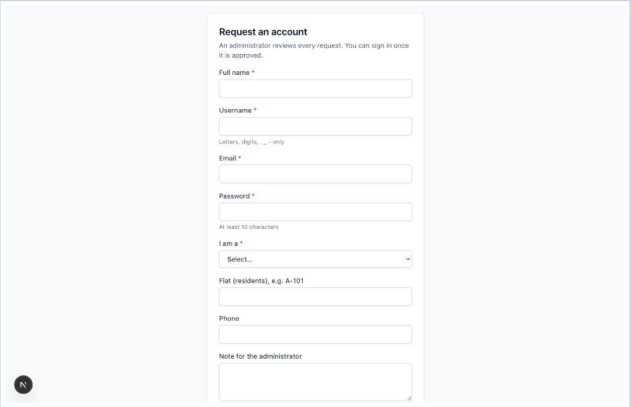
4.5 Other protections

Threat	Protection
SQL injection	All MySQL access goes through Prisma, which sends values as parameters, never pasted into SQL text
MongoDB query injection	Search text is escaped before being used in a regular expression; ids are parsed and validated
Cross-site request forgery	Origin header check on every change plus SameSite=Lax cookies
Guessing passwords	Database-backed throttle (5 per 15 minutes per identifier) and slow bcrypt hashing
Finding out who has an account	Same error message and same timing for wrong user and wrong password
Spamming the sign-up form	5 requests per hour per address, stored in MySQL
Leaking internals	Errors return a code and a plain message; logs record only the error type, never connection strings or queries
Seeing other people's data	Every list and detail endpoint filters by role and ownership on the server (section 5)
Racing requests	Conditional updates and unique indexes: two admins approving one request, or two cars claiming one slot, cannot both succeed

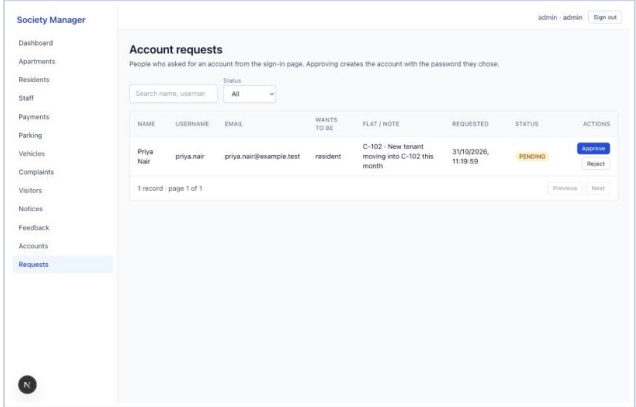
4.6 Requesting an account and admin approval

New people do not get an account automatically. They fill in a public form; nothing can sign in until an administrator approves it.

1. The person opens **Request an account** from the sign-in page and gives name, username, email, password, role (resident, maintenance or security), flat and an optional note.
2. The server validates the form, applies the 5-per-hour limit, and refuses a username or email that already has an account or a pending request. Someone who already has a login and needs a second flat is told to ask the administrator to link it.
3. The password is hashed immediately and the request is saved in the `AccountRequest` table with status PENDING.
4. An administrator sees the request under **Requests** (and a count on the dashboard), then **Approves**, choosing the role and the resident or staff record it belongs to, or **Rejects** with an optional reason.
5. Approval runs in one database transaction: it claims the request (so a second reviewer gets a conflict), checks the username is still free, creates the account with the requester's own hashed password, links it, and records who approved it and when.



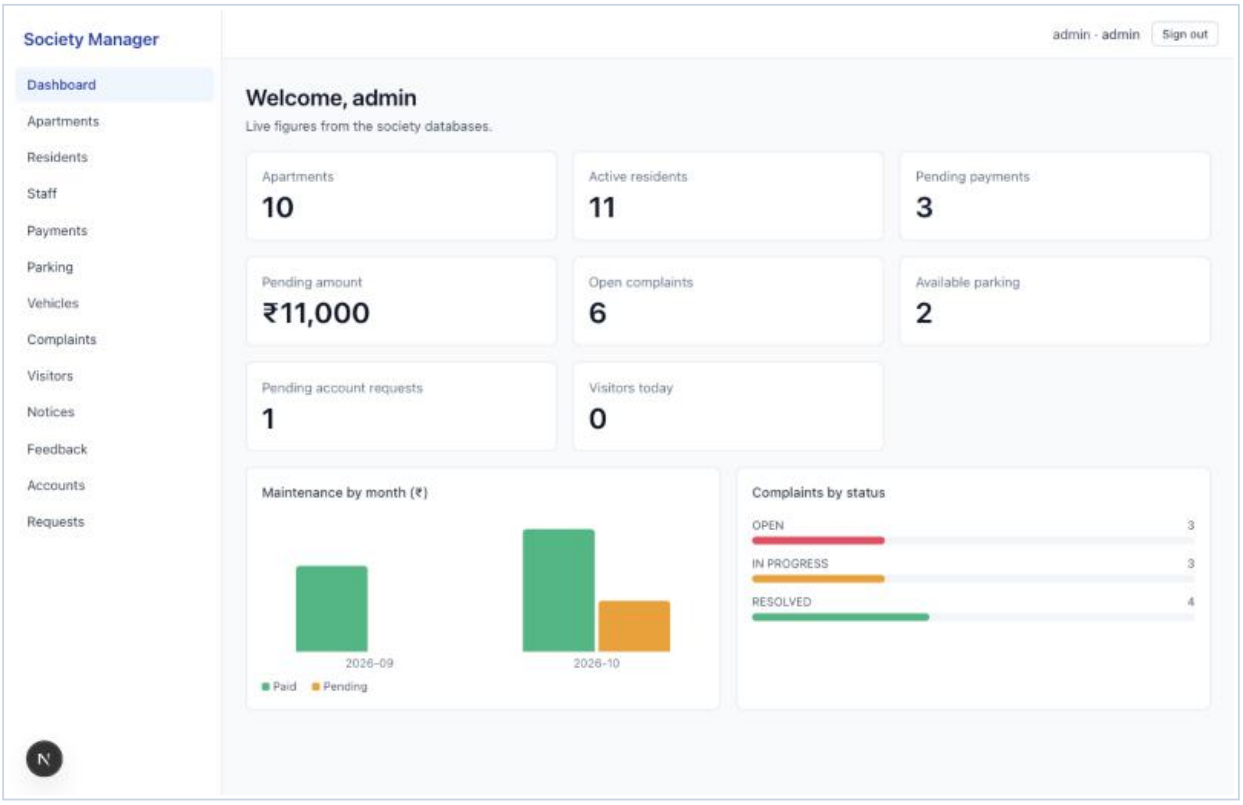
The public request form



The admin's Requests page with a pending request

5. Roles and what each person sees

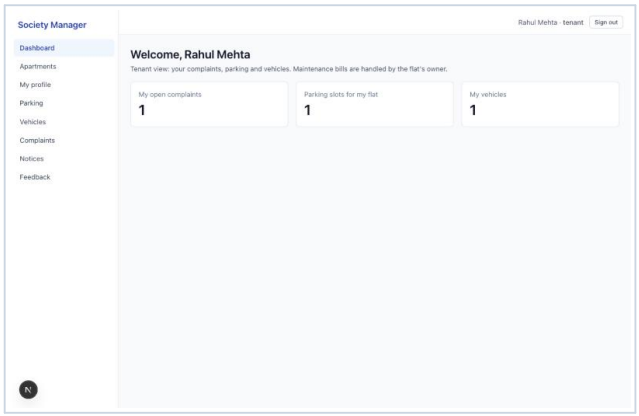
Role	Can see and do	Cannot
Administrator	All records and settings; create and approve accounts; billing; assign complaints; notices; feedback review	Deactivate their own account
Owner	Bills and pending balance for every flat they own; everyone registered in those flats; parking and vehicles; own complaints and feedback; owner notices	See other flats, staff, visitors or accounts
Tenant	Their own profile; parking and their own vehicles; own complaints and feedback; tenant notices	See any maintenance bills or neighbours
Maintenance	Complaints assigned to them; mark work resolved; add notes; notices	See residents, bills, parking
Security	Register visitors and record exits; search visitor history; pick a host resident; notices	See bills, complaints or resident details



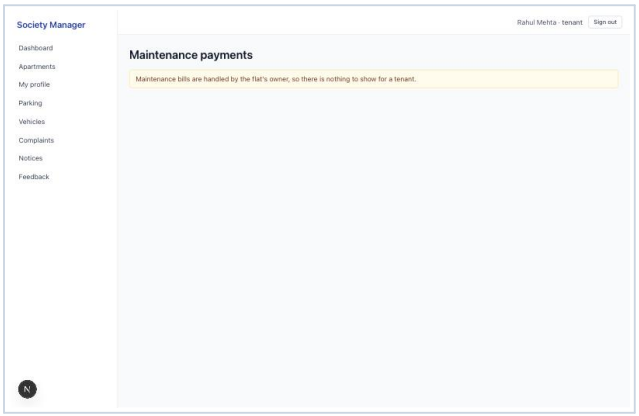
Administrator dashboard: live counts from both databases, maintenance collected vs pending by month, complaints by status.

5.1 Owners and tenants

Residents are split by their relationship to the flat. Owners are billed, so they see bills and who lives in their flats. Tenants are not billed, so the Payments page is not in their menu, and if they type the address anyway the server refuses and the page explains why. The header shows "owner" or "tenant" next to the name.



Tenant dashboard: complaints, parking, vehicles; no bills

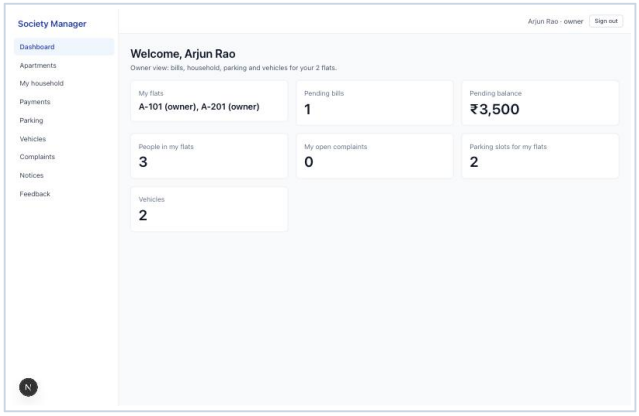


A tenant who opens /payments directly: refused by the server, explained on the page

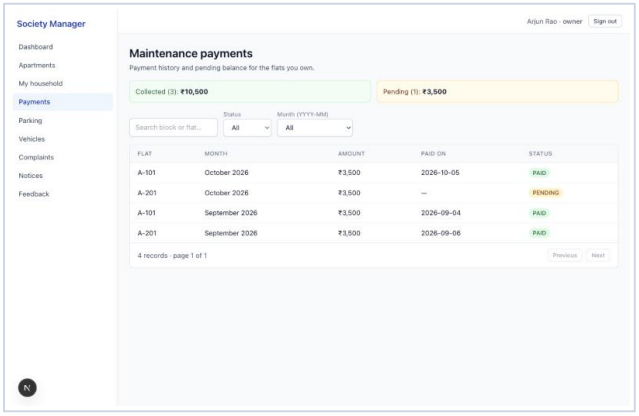
5.2 One person with several flats

People often own more than one flat, or own one and rent another. The database stores one **resident row per person per flat**, and all of a person's rows point at the same login through `Resident.account_id`. Access is then worked out flat by flat:

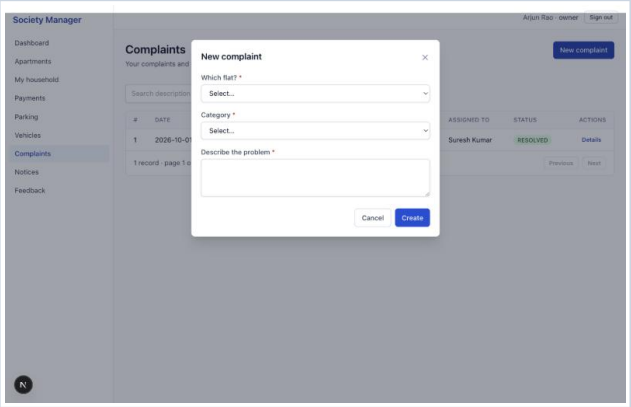
- **Bills and household** only for the flats where they are the owner.
- **Complaints, feedback, parking and their own vehicles** across all their flats.
- **Notices** for every audience that applies: someone who owns one flat and rents another reads both owner and tenant notices.
- When they file a complaint or feedback, the form asks **which flat** it is about, and the server refuses a flat that is not theirs.
- An admin adds a flat to an existing login with **Accounts → Link another flat**. One login can have only one row per flat (a unique index enforces it).



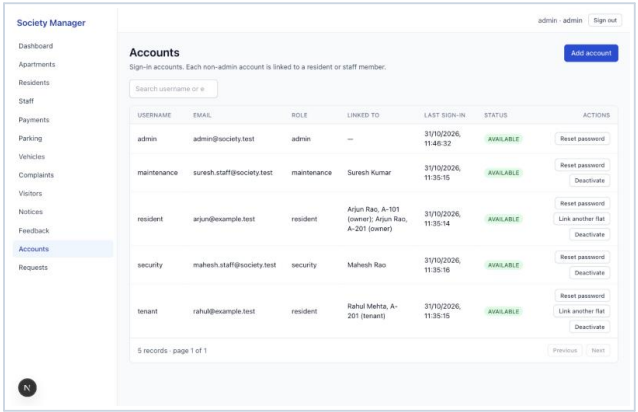
Arjun owns A-101 and A-201: one login, both flats



His Payments page lists bills for both flats he owns



With two flats, a new complaint asks "Which flat?"



Admin Accounts page: Arjun's login linked to both flats; Link another flat

6. The data model

Data that needs strict relationships lives in MySQL; event-like data that grows over time lives in MongoDB. MongoDB documents refer to MySQL rows by their integer ids (for example a visitor's `apartmentId`).

MySQL tables

Table	Holds	Key rules
Apartment	One row per flat: block, floor, flat number, BHK	Unique (block, flat_no)
Resident	One person in one flat, OWNER or TENANT	Optional account_id; unique (account_id, apartment_id); deactivated, not deleted
Staff	Staff profile, role, phone, salary	Salary is DECIMAL(10,2)
ParkingSpace	A slot, its vehicle type, AVAILABLE or ALLOCATED	Unique slot number
Vehicle	A resident's car or bike, optionally parked	Unique vehicle number; unique parking_id, so one car per slot
MaintenancePayment	Monthly bill for a flat: amount, month, PAID or PENDING	Unique (apartment_id, payment_month); amount DECIMAL(10,2)
Complaint	Raised by a resident, optionally assigned to staff	Status OPEN / IN_PROGRESS / RESOLVED
UserAccount	Login: username, email, bcrypt hash, role	Unique username and email; sessions_valid_after for revocation
AccountRequest	Public sign-up requests and their review	Who reviewed, when, and which account it produced
LoginAttempt	Failed sign-in and sign-up counters	Shared throttle across processes

Relationships: an apartment has many residents, parking slots and bills; a resident has many vehicles and complaints; a staff member has many assigned complaints; a login has many resident rows (one per flat) or one staff row. Records with history are deactivated rather than deleted, and foreign keys refuse deletes that would orphan bills or complaints.

MongoDB collections

Collection	Holds	Notes
visitors	Name, purpose, flat, host resident, vehicle, entry and exit time	Exit can be recorded only once
notices	Notices and announcements with audience, priority, expiry	Non-admins see only published, unexpired notices for their audience
feedback	Resident feedback with rating and admin status	Tied to the flat it was filed from
complaint_activity	Append-only timeline: created, assigned, status changed, notes	Shown to the resident as a history

When the two databases disagree

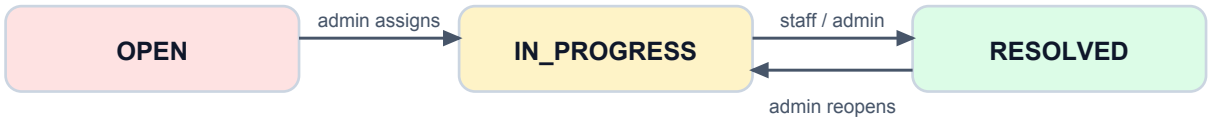
Some actions write to both: changing a complaint's status updates MySQL and appends to MongoDB. If MongoDB is down, the MySQL change is kept and the reply is 207 Partial Success with a clear warning, instead of pretending everything worked.

Migrations

Migration	Purpose
20261030000000_init	Full initial schema
20261030010000_session_revocation	Adds sessions_valid_after so sign-out revokes tokens
20261030020000_login_throttle	Adds the shared LoginAttempt table
20261031000000_account_requests	Adds AccountRequest for sign-up and approval
20261031010000_multi_flat_residents	Moves the login link onto Resident so one login covers several flats; copies existing links before dropping the old column

7. Key workflows

Complaints



Each change also appends an event to MongoDB complaint_activity, which the resident sees as a timeline.

A resident files a complaint (choosing the flat if they have several). An admin assigns it to a staff member, which moves it to IN_PROGRESS. The staff member or an admin marks it RESOLVED; an admin can reopen it. Status changes use guarded updates, so two people editing at once cannot overwrite each other. If a staff member is deactivated, their in-progress complaints go back to OPEN and unassigned, with a timeline note.

Maintenance billing

- One bill per flat per month; a second bill for the same month is refused with 409.
- A PAID bill needs a payment date, a PENDING bill must not have one, and the payment date cannot be in the future.
- Amounts are stored as exact decimals, never floating point.

Parking

- A slot is for a car or a bike, and only a matching vehicle can be allocated to it.
- Allocation is a conditional update ("only if still AVAILABLE") plus a unique index on the vehicle's slot, so two simultaneous allocations cannot both win. The tests fire two at once and check that exactly one succeeds.
- When a resident moves out, their slots are released and vehicles retired in the same transaction; moving a resident to another flat while their car is parked is refused until the slot is released.

Visitors

Security registers an entry with the destination flat and, optionally, the host resident (the list only shows people in that flat). Recording the exit flips the status once; a second exit for the same visit is refused.

Notices

Admins write notices as drafts, publish and archive them, and target ALL, OWNERS, TENANTS, MAINTENANCE or SECURITY. Expired notices disappear for everyone except admins.

8. Edge cases handled

Each row below is covered by an automated test.

Situation	What happens
Owner of A asks for the bills of flat B they only rent	Nothing returned; the single-bill endpoint gives 403
Tenant opens their flat's detail page	Flat only: no payment summary, no neighbour list
Multi-flat person files a complaint without naming a flat	Refused (400) with "choose which flat"; a flat that is not theirs gives 403
Same person, same phone and email in two flats	Allowed: contact details are not unique per resident row
Linking a login to a second row in a flat it already has	Refused (409), also enforced by a unique index
Resident moves out	Parking released, vehicles retired; login locked only if it was their last flat
Unlinking a login's last active flat	Refused: deactivate the account instead
Staff member deactivated with open work	Their in-progress complaints return to the queue with a timeline entry
Duplicate monthly bill	Refused (409)
Payment dated in the future	Refused (400)
Bike into a car slot, or an already-taken slot	Refused (400 / 409)
Two allocations or two approvals at the same moment	Exactly one succeeds
Visitor exit recorded twice	Second attempt refused (409)
Sign-out in the same second as sign-in	Still revoked (millisecond timestamps)
Password reset	All existing sessions for that login stop working
Sixth wrong password in 15 minutes	Refused (429) until the window passes

9. Testing and quality checks

Check	Result on 31 October 2026
npm run lint	No warnings or errors
npm run typecheck	No errors
npm run build	Production build succeeds
npm test	21 unit tests pass: status transitions, notice audiences, payment rules, validation, session tokens, environment checks
npm run test:integration	18 tests pass against live MySQL and MongoDB, including every edge case in section 8
Fresh database	All five migrations, the seed and the integration tests also pass on a brand-new empty database
GitHub Actions CI	On every push: lint, type-check, unit tests, build, database setup and integration tests in MySQL and MongoDB containers

Integration tests call the real route handlers with real database connections. Every row they create is tagged and removed afterwards, so they can run against a database that also holds demo data.

10. Known limitations

- One role per login: a staff member who also lives in the society needs two logins.
- Reactivating a resident does not reopen a login that was locked when they moved out; an admin turns it back on in Accounts.
- Dropdowns load up to 5,000 records with a filter box; a very large society would want server-side search pickers.
- The sign-up limit identifies clients by the X-Forwarded-For header; without a trusted proxy in front of the app, a client can send a fake value to get around it.
- No payment gateway, email or SMS: bills are marked paid by the administrator.
- Writes that touch both databases are not atomic across them; partial failures are reported (207) rather than hidden.
- This is an academic project and is not claimed to meet any formal security or compliance standard.

Appendix: API at a glance

Area	Endpoints	Who
Auth	POST /api/auth/login, POST /api/auth/logout, GET /api/auth/session, POST /api/auth/request-account	Public / signed in
Account requests	GET /api/account-requests, PATCH /api/account-requests/:id (approve, reject)	Admin
Accounts	GET, POST /api/accounts; PATCH /api/accounts/:id (isActive, password, linkResidentId, unlinkResidentId)	Admin
Apartments	GET, POST /api/apartments; GET, PATCH, DELETE /api/apartments/:id	Admin; residents read their flats
Residents	GET, POST /api/residents; GET, PATCH, DELETE /api/residents/:id; GET /api/residents/hosts	Admin; owners see household; security uses hosts
Payments	GET, POST /api/payments; GET /api/payments/summary; GET, PATCH /api/payments/:id	Admin; owners for owned flats
Parking, vehicles	GET, POST and GET, PATCH, DELETE on /api/parking and /api/vehicles	Admin; residents read their own
Complaints	GET, POST /api/complaints; GET, PATCH /api/complaints/:id; POST .../activity	Admin, residents, maintenance
Visitors	GET, POST /api/visitors; PATCH /api/visitors/:id (exit)	Admin, security
Notices, feedback	GET, POST and PATCH on /api/notices and /api/feedback	Everyone reads notices; residents submit feedback
Dashboard	GET /api/dashboard	Everyone; figures depend on role

Status codes: 200/201 success, 207 partial success, 400 invalid input, 401 not signed in, 403 not allowed, 404 not found, 409 conflict, 429 too many attempts, 500/503 server or database problem.